

实验报告

实验 6 单周期 CPU 设计与测试

课程名称

数字逻辑与计算机组成

指导教师

吴海军

1 实验目的

1. 掌握 RV32I 控制器的设计方法。
2. 掌握多输入多输出组合逻辑部件设计方法。
3. 掌握最小风险法设计方法。
4. 掌握 RV32I 单周期 CPU 的设计方法。
5. 掌握加载程序初始地址和结束程序执行的设计方法。
6. 掌握计算程序执行周期的设计方法。
7. 理解冯诺依曼程序存储和程序控制的思想。
8. 掌握数据存储器的访问方法。
9. 掌握 RV32I 汇编程序编译方法。
10. 掌握单周期 CPU 加载程序执行的方法。
11. 掌握 RARS 编译仿真工具的使用方法。
12. 掌握冒泡排序算法的设计方法。
13. 掌握数据存储器数据交换的设计方法。
14. 前后课程实验内容贯通。
15. 掌握代码段和数据段统一编址后的代码文件和数据文件加载方法。
16. 理解 C 语言程序、RISC-V 汇编程序与机器语言代码之间的对应关系。

2 实验环境

- 平台：Logisim。
- 辅助工具：RARS。

3 实验内容

3.1 控制器设计实验

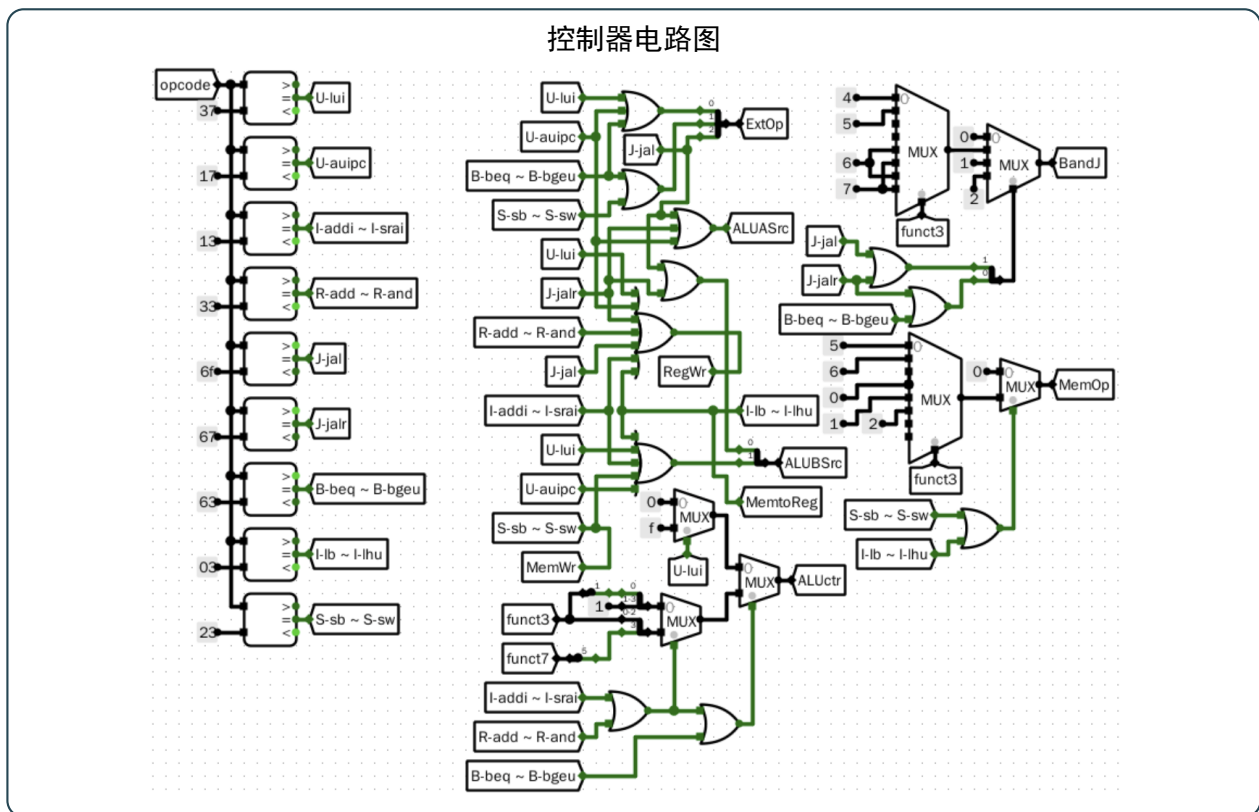
3.1.1 实验整体方案设计

- 在 Logisim 中创建“控制器”子电路，根据 opcode、func3 和 func7[5] 生成 ExtOp、RegWr、ALUASrc、ALUBSrc、ALUctr、MemOp、MemWr、BandJ、MemtoReg、NPCASrc 和 NPCBSrc 等信号。
- 具体的指令-控制信号对应关系可参照实验讲义中的表 6.1。

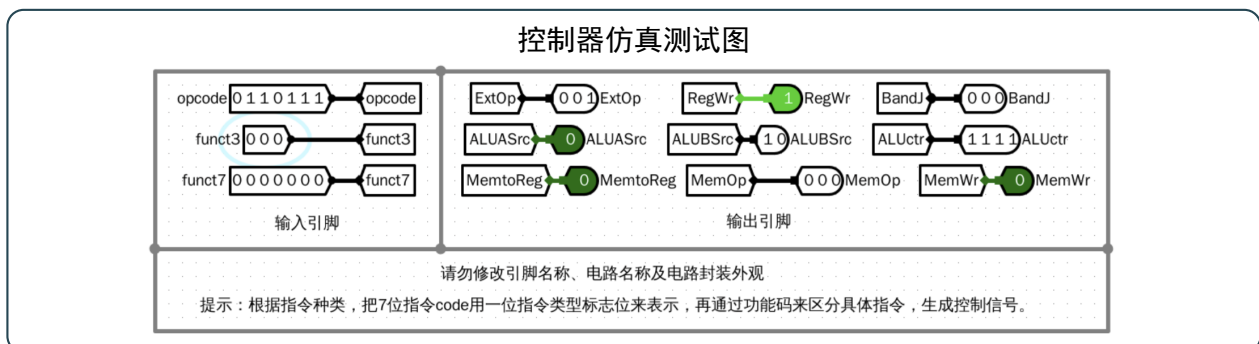
3.1.2 控制信号说明

控制信号	位宽	作用与取值说明
ExtOp	3	立即数产生器的控制信号，根据指令类型赋值。
RegWr	1	寄存器堆写使能控制信号，根据指令类型在相应的执行阶段赋值，高电平有效。
ALUASrc	1	ALU 输入端 A 的选择信号，0: R[Rs1]，1: PC。
ALUBSrc	2	ALU 输入端 B 的选择信号，00: R[Rs2]，01: 4，10: imm。
ALUctr	4	ALU 控制信号，根据指令操作码和功能码赋值。
MemOp	3	数据存储器读写操作以及字节长度控制信号。
MemWr	1	数据存储器写使能控制信号，高电平有效。
BandJ	3	用于分支跳转控制器的输入，生成下地址逻辑部件中控制信号 NPCASrc 和 NPCBSrc。
MemtoReg	1	寄存器堆写入数据来源控制信号，0: ALU 运算结果，1: 数据存储器读取输出。
NPCASrc	1	下地址逻辑中加法器输入端 A 的选择信号，0: PC，1: R[Rs1]。
NPCBSrc	1	下地址逻辑中加法器输入端 B 的选择信号，0: 4，1: imm。

3.1.3 实验原理图和电路图

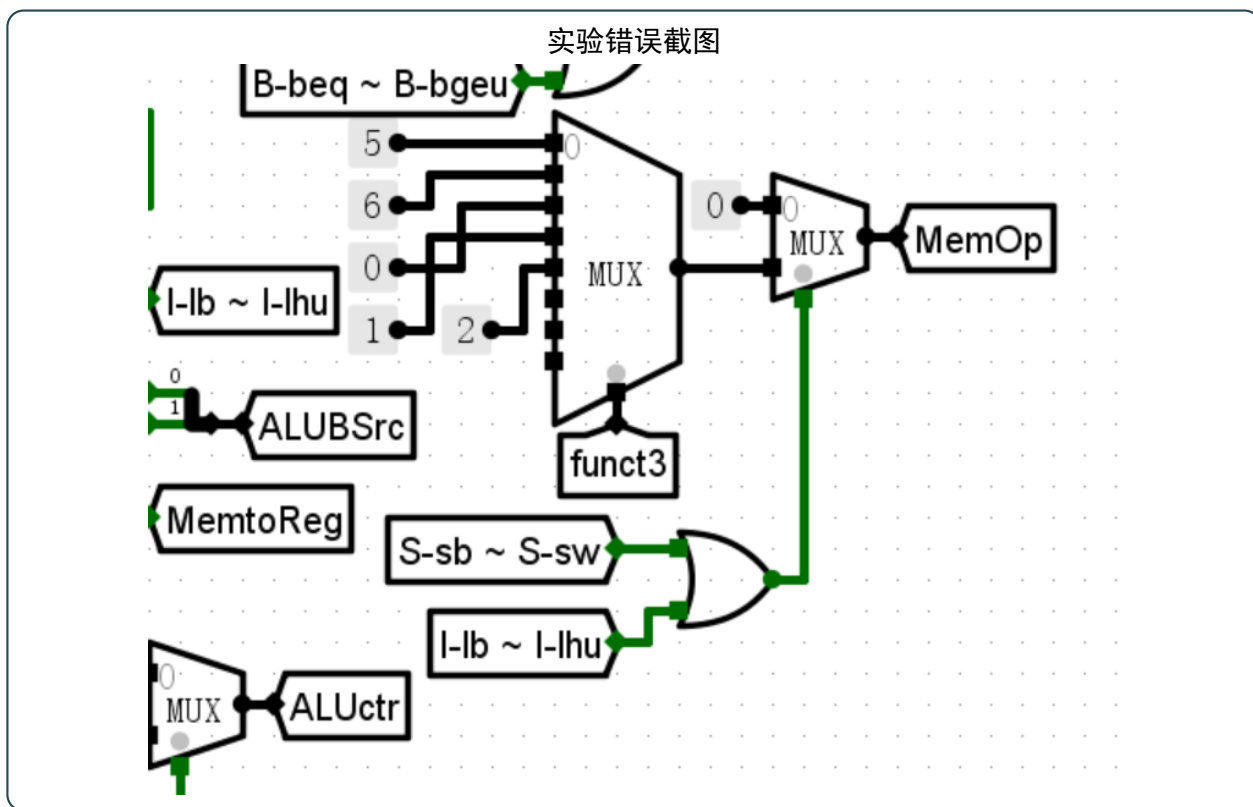


3.1.4 实验数据仿真测试图



3.1.5 错误现象及分析

对于指令 lbu/lhu 的对应的 MemOp 信号，连接错误，导致输出错误。正确连接后成功通过了测试数据。

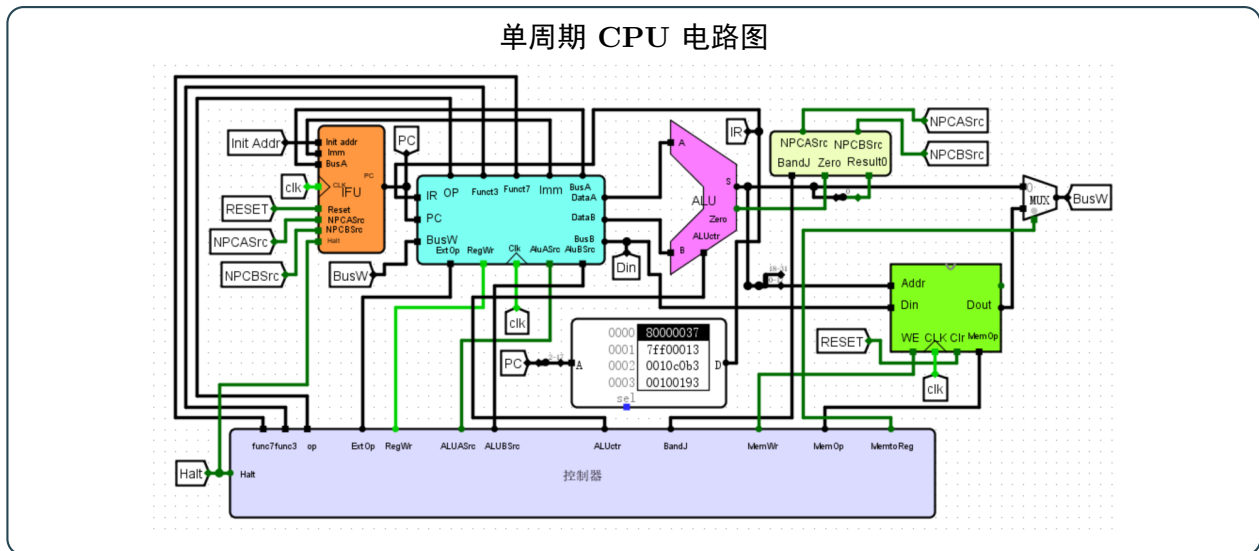
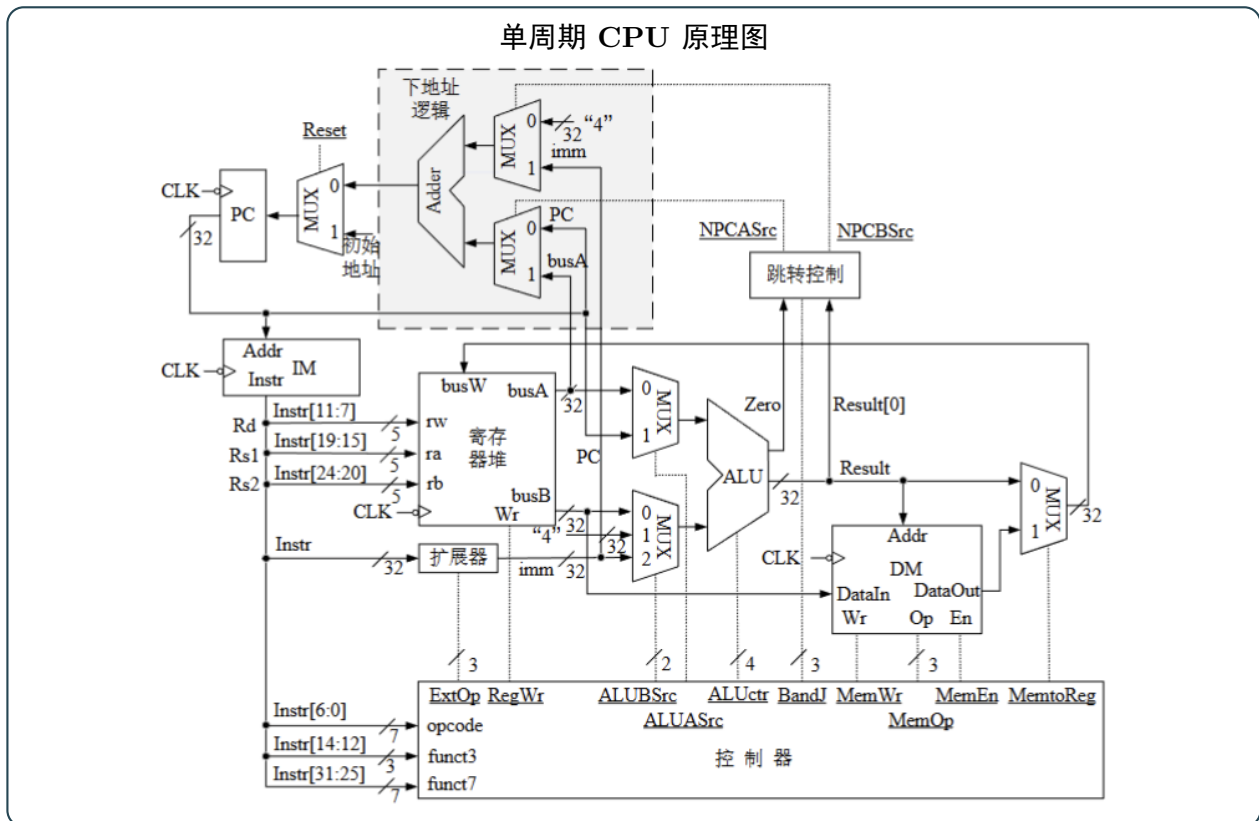


3.2 单周期 CPU 设计实验

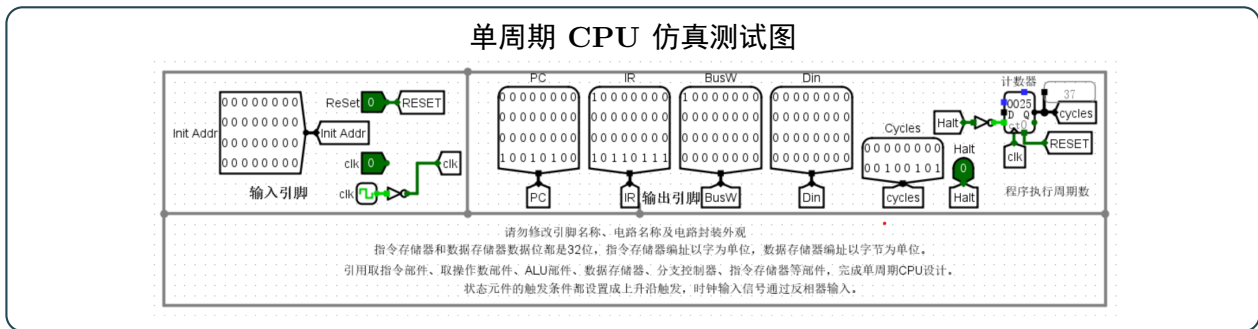
3.2.1 实验整体方案设计

- 在实验五数据通路部件基础上加入控制器部件，连接成单周期处理器。
- PC 寄存器、寄存器堆和数据存储器的时钟边沿、Reset 复位与 Halt 停机信号都需要正确处理。

3.2.2 实验原理图和电路图



3.2.3 实验数据仿真测试图



3.2.4 错误现象及分析

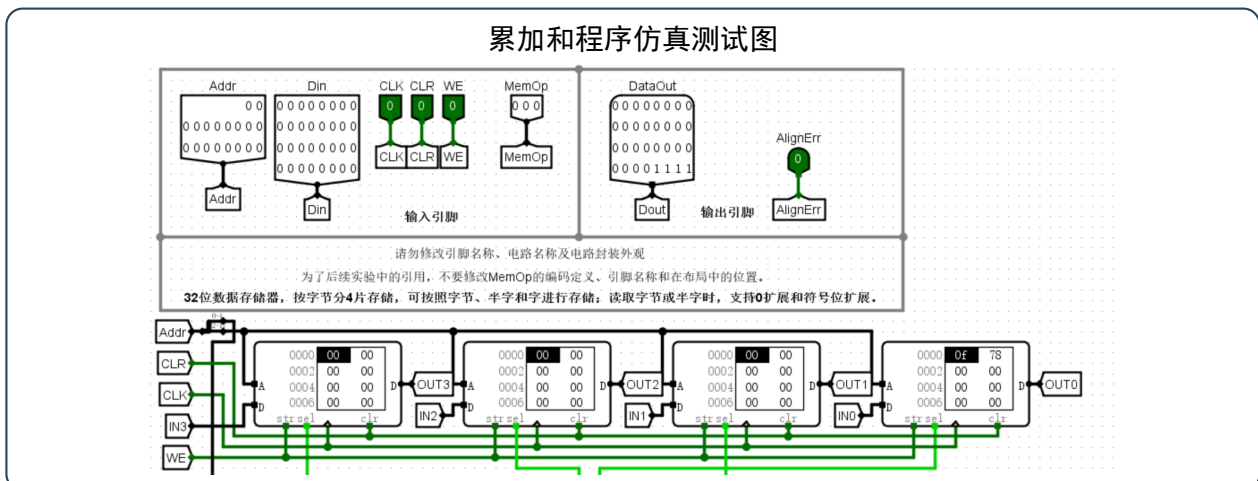
错误来源于模块封装时引脚错误与加载了错误的指令镜像文件，导致测试数据无法通过。正确连接引脚并加载正确的指令镜像后成功通过了测试数据。

3.3 累加和程序测试实验

3.3.1 实验整体方案设计

- 使用 lab6.3.asm 编写累加和程序，并将导出的机器代码加载到指令存储器中。
- 数据存储器地址 0 号单元存放参数 n ，地址 4 号单元存放结果 S 。
- 程序结束后应通过 `ecall` / Halt 停止执行。

3.3.2 实验数据仿真测试图



3.3.3 错误现象及分析

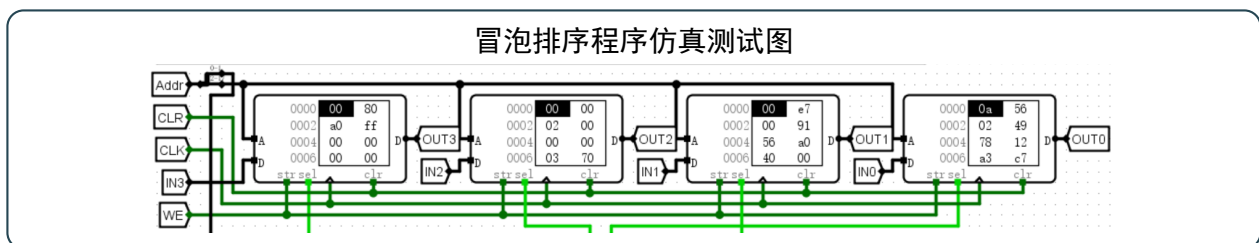
实验中没有出现错误。

3.4 冒泡排序程序测试实验

3.4.1 实验整体方案设计

- 使用 lab6.4.asm 编写冒泡排序程序，并将导出的机器代码和数据镜像分别加载到指令存储器和数据存储器中。
- 程序运行结束后检查排序结果是否与预期一致。

3.4.2 实验数据仿真测试图



3.4.3 错误现象及分析

实验中没有出现错误。

3.5 官方测试集实验

3.5.1 实验整体方案设计

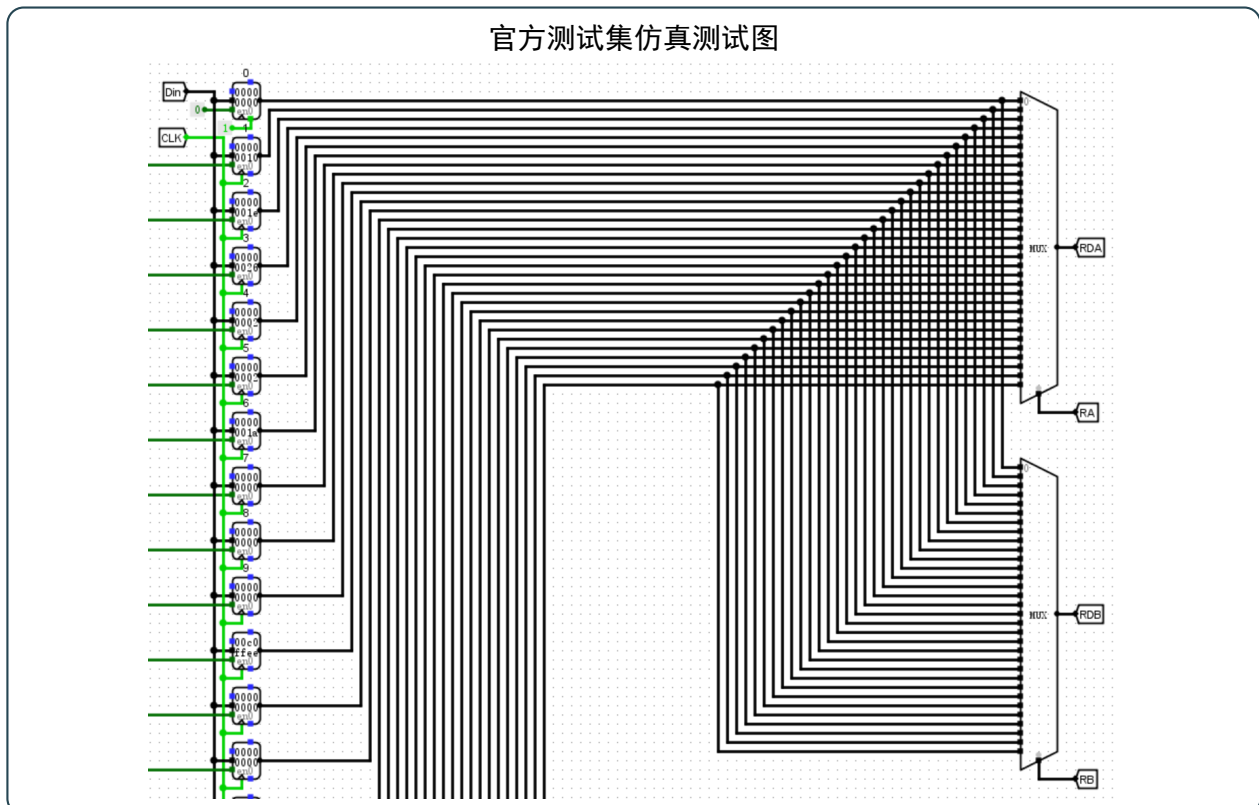
- 使用实验讲义提供的 rv32ui-p 测试程序和对应的指令镜像、数据镜像进行验证。
- 按照测试结果记录 x1、x3、x10 寄存器值和时钟周期数。

3.5.2 官方测试集验证表

序号	测试指令	x1 寄存器输出值	x3 寄存器输出值	x10 寄存器输出值	时钟周期数
1	add	0x0010	0x0026	0xc0ffee	458
2	addi	0x0021	0x0019	0xc0ffee	235
3	and	0x11111111	0x001b	0xc0ffee	478
4	andi	0x00ff00ff	0x000e	0xc0ffee	191
5	auipc	0	0x0003	0xc0ffee	52
6	beq	0x0003	0x0015	0xc0ffee	284
7	bge	0x0003	0x0018	0xc0ffee	302
8	bgeu	0x0003	0x0018	0xc0ffee	327
9	blt	0x0003	0x0015	0xc0ffee	284
10	bltu	0x0003	0x0015	0xc0ffee	309
11	bne	0x0003	0x0015	0xc0ffee	284
12	jal	0x0003	0x0003	0xc0ffee	48
13	jalr	0	0x0007	0xc0ffee	108

序号	测试指令	x1 寄存器输出值	x3 寄存器输出值	x10 寄存器输出值	时钟周期数
14	lb	0x2000	0x0013	0xc0ffee	238
15	lbu	0x2000	0x0013	0xc0ffee	238
16	lh	0x2000	0x0002	0xc0ffee	43
17	lhu	0x2000	0x0002	0xc0ffee	48
18	lui	0xffff800	0x0006	0xc0ffee	58
19	lw	0x2000	0x0013	0xc0ffee	260
20	or	0x11111111	0x001b	0xc0ffee	481
21	ori	0x00ff00ff	0x000e	0xc0ffee	198
22	sb	0x0001	0x0017	0xc0ffee	423
23	sh	0x2000	0x0004	0xc0ffee	65
24	sll	0x0400	0x002b	0xc0ffee	486
25	slli	0x0021	0x0019	0xc0ffee	234
26	slt	0x0010	0x0026	0xc0ffee	452
27	slti	0x00ff00ff	0x0019	0xc0ffee	230
28	sltiu	0x00ff00ff	0x0019	0xc0ffee	230
29	sltu	0x0010	0x0026	0xc0ffee	452
30	sra	0x0400	0x002b	0xc0ffee	505
31	srai	0x0021	0x0019	0xc0ffee	249
32	srl	0x0400	0x002b	0xc0ffee	499
33	srli	0x0021	0x0019	0xc0ffee	243
34	sub	0x0010	0x0025	0xc0ffee	450
35	sw	0x12233001	0x0017	0xc0ffee	483
36	xor	0x11111111	0x001b	0xc0ffee	480
37	xori	0x00ff00ff	0x000e	0xc0ffee	200

3.5.3 实验数据仿真测试图



3.5.4 错误现象及分析

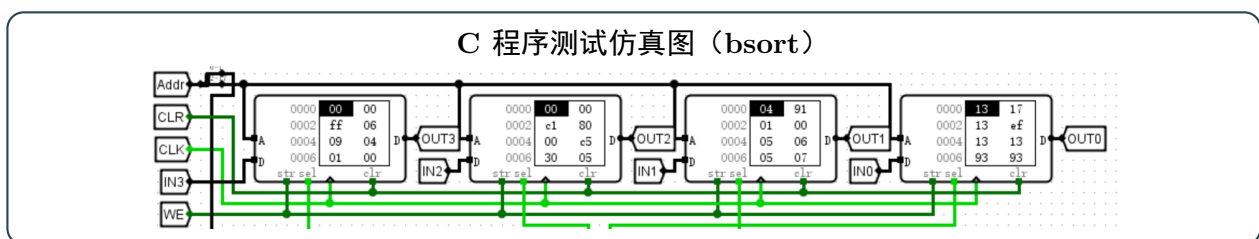
实验中没有出现错误。

3.6 计算机系统基础 PA 程序测试实验

3.6.1 实验整体方案设计

- 本部分用于验证后续课程中的 C 程序测试，也可以作为扩展测试参考。
- 根据需要配置交叉编译环境、测试目录和镜像文件加载方式。

3.6.2 实验数据仿真测试图



3.6.3 错误现象及分析

实验中没有出现错误。

4 思考题

4.1 思考题 1

如何在单 CPU 上实现多任务处理，例如同时执行计算累加和与数据排序两个程序，阐述思路。

通过保存和恢复程序上下文 (PC、通用寄存器和必要状态)，让两个程序按时间片轮流执行。调度器在时钟中断或任务主动让出时切换上下文，恢复后即可继续运行上一次停下的位置。单 CPU 不能真正并行，只能依靠这种交替运行来实现多任务，并提高系统的响应性。

4.2 思考题 2

在 CPU 的基础上，如何实现键盘输入、TTY 输出部件等输入输出设备的数据访问，构建完整的计算机系统？

采用内存映射 I/O，将键盘和 TTY 等设备的状态寄存器、数据寄存器映射到固定地址。CPU 通过地址译码把访存请求分发给对应外设，再按读写协议完成数据交换，并根据设备忙闲状态决定是否等待。配合中断机制和驱动程序，就能把输入输出设备纳入统一的计算机系统。

4.3 思考题 3

阐述如何在单周期 CPU 基础上实现多周期 CPU。

将单周期中的取指、译码、执行、访存、回写拆分到多个时钟周期中完成。增加控制状态机和中间寄存器后，各阶段可以按顺序复用 ALU 和存储器资源，并通过控制信号逐步推进指令执行。这样能缩短单个周期的组合逻辑路径，提升最高工作频率。

4.4 思考题 4

阐述如何在单周期 CPU 基础上实现流水线 CPU。

在多周期分解的基础上加入流水线寄存器，让多个指令的不同阶段重叠执行。通常可划分为取指、译码、执行、访存、回写等阶段，并在阶段之间插入寄存器。再通过前递、暂停和冲刷水处理数据相关、控制相关等冒险问题，保证各级结果能够正确传递。